

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“Jnana Sangama”, Machhe, Belagavi, Karnataka-590018



Lab Experiment Record

Project Management with Git [BCSL358C]

Submitted in partial fulfillment towards AEC of 3rd semester of

**Bachelor of Engineering
in
Computer Science and Engineering
(Artificial Intelligence & Machine Learning)**

Submitted by

**RIA KUZHIKANDATHIL CLIMIS
4GW24CI034**



**DEPARTMENT OF CSE (Artificial Intelligence & Machine Learning)
GSSS INSTITUTE OF ENGINEERING & TECHNOLOGY FOR WOMEN
(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi & Govt. of Karnataka)
K.R.S ROAD, METAGALLI, MYSURU-570016, KARNATAKA
(Accredited by NAAC)**

2025-2026

Project Management with Git		Semester	3
Course Code	BCS358C	CIE Marks	50
Teaching Hours/Week (L:T:P: S)	0: 0 : 2: 0	SEE Marks	50
Credits	01	Exam Marks	100
Examination type (SEE)	Practical		
Course objectives: • .To familiar with basic command of Git • To create and manage branches • To understand how to collaborate and work with Remote Repositories • To familiar with version controlling commands			
SLNO	Experiments		
1	Setting Up and Basic Commands Initialize a new Git repository in a directory. Create a new file and add it to the staging area and commit the changes with an appropriate commit message.		
2	Creating and Managing Branches Create a new branch named "feature-branch." Switch to the "master" branch. Merge the "feature-branch" into "master."		
3	Creating and Managing Branches Write the commands to stash your changes, switch branches, and then apply the stashed changes.		
4	Collaboration and Remote Repositories Clone a remote Git repository to your local machine.		
5	Collaboration and Remote Repositories Fetch the latest changes from a remote repository and rebase your local branch onto the updated remote branch.		
6	Collaboration and Remote Repositories Write the command to merge "feature-branch" into "master" while providing a custom commit message for the merge.		
7	Git Tags and Releases Write the command to create a lightweight Git tag named "v1.0" for a commit in your local repository.		
8	Advanced Git Operations		

	Write the command to cherry-pick a range of commits from "source-branch" to the current branch.
9	Analysing and Changing Git History Given a commit ID, how would you use Git to view the details of that specific commit, including the author, date, and commit message?
10	Analysing and Changing Git History Write the command to list all commits made by the author "JohnDoe" between "2023-01-01" and "2023-12-31."
11	Analysing and Changing Git History Write the command to display the last five commits in the repository's history.
12	Analysing and Changing Git History Write the command to undo the changes introduced by the commit with the ID "abc123".
Course outcomes (Course Skill Set): At the end of the course the student will be able to: • Use the basics commands related to git repository • Create and manage the branches • Apply commands related to Collaboration and Remote Repositories • Use the commands related to Git Tags, Releases and advanced git operations • Analyse and change the git history	

Experiment 1: Setting Up and Basic Commands

A new Git repository was initialized in a directory, and a file as created, staged, and committed with a suitable commit message.

```
4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (master)
$ mkdir git1

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (master)
$ git init
Reinitialized existing Git repository in C:/Users/4gw19/OneDrive/Desktop/git-lab-clean/.git/

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (master)
$ touch file1.txt

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (master)
$ git add file1.txt

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (master)
$ git commit -m"initial commit"
[master (root-commit) f0de87d] initial commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 file1.txt
```

The Git repository was initialized successfully and the file was committed without any errors.

The mkdir command is used to create a new directory (folder) in the system. The commands git init, git add, and git commit are used to initialize the repository, add files to the staging area, and save changes respectively.

Experiment 2: Creating and Managing Branches

A new branch named feature-branch was created. The user then switched back to the master branch and merged the feature-branch into it.

```
4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (master)
$ git branch feature-branch

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (master)
$ git checkout master
Already on 'master'

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (master)
$ git checkout feature-branch
Switched to branch 'feature-branch'

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (feature-branch)
$ git switch master
Switched to branch 'master'

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (master)
$ git merge feature-branch
Already up to date.
```

The changes were merged successfully between the branches.

The checkout command is used to move between branches; alternatively, the switch command can also be used. The merge command is applied to combine changes from two existing branches.

Experiment 3: Creating and Managing Branches

Commands were written to stash the current changes, switch to another branch, and apply the stashed changes.

```
4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (master)
$ git stash
Saved working directory and index state WIP on master: f0de87d initial commit
```

```
4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (feature-branch)
$ git stash apply
On branch feature-branch
changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file1.txt

no changes added to commit (use "git add" and/or "git commit -a")

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (feature-branch)
$ git status
On branch feature-branch
changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file1.txt

no changes added to commit (use "git add" and/or "git commit -a")

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (feature-branch)
$ git add file1.txt

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (feature-branch)
$ git commit -m"a"
[feature-branch 55d4912] a
 1 file changed, 1 insertion(+)

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (feature-branch)
$ git status
On branch feature-branch
nothing to commit, working tree clean

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (feature-branch)
$ git stash apply
On branch feature-branch
nothing to commit, working tree clean
```

The changes were stashed and later applied successfully on another branch.

Before applying the stash, the modified files had to be committed. The commands stash and stash apply were used to temporarily save and reapply the changes.

Experiment 4: Collaboration and Remote Repositories

A remote Git repository was cloned to the local system.

```
4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (feature-branch)
$ git clone https://github.com/ina528/4GW24CI015.git
Cloning into '4GW24CI015'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

The remote repository was cloned successfully.

The clone command is used along with the repository URL obtained from GitHub to create a local copy of the remote repository.

Experiment 5: Collaboration and Remote Repositories

The latest updates from a remote repository were fetched and the local branch was rebased onto the updated remote branch.

```
4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (feature-branch)
$ git fetch origin
fatal: 'origin' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (feature-branch)
$ git remote -v

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (feature-branch)
$ git remote add origin https://github.com/ina528/4GW24CI015.git

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (feature-branch)
$ git remote -v
origin https://github.com/ina528/4GW24CI015.git (fetch)
origin https://github.com/ina528/4GW24CI015.git (push)

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (feature-branch)
$ git fetch origin
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 869 bytes | 57.00 KiB/s, done.
From https://github.com/ina528/4GW24CI015
* [new branch]      main      -> origin/main
```

The local branch was successfully rebased with the latest changes from the remote branch.

Initially, the fetch operation failed due to the absence of a configured remote repository. After adding the remote repository, the fetch and rebase operations were completed successfully.

```
4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (master)
$ git rebase origin/main
Successfully rebased and updated refs/heads/master.
```


Experiment 6: Collaboration and Remote Repositories

The feature-branch was merged into the master branch using a custom commit message.

```
4gw19@LAPTOP-NGUM35FU MINGW64 ~/gitlab (master)
$ git checkout master
Already on 'master'
Your branch is up to date with 'origin/master'.

4gw19@LAPTOP-NGUM35FU MINGW64 ~/gitlab (master)
$ git merge feature-branch -m"merged into master"
Already up to date.
```

The branches were merged successfully with the specified commit message.

The merge command was used to combine the branches.

Experiment 7: Git Tags and Releases

A lightweight Git tag named v1.0 was created for a commit in the local repository.

```
4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (feature-branch)
$ git tag v1.0

4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (feature-branch)
$ git tag
v1.0
```

The lightweight tag v1.0 was created successfully.

The tag command is used to create tags and can also be used to list all existing tags in the repository.

Experiment 8: Advanced Git Operations

A range of commits was cherry-picked from the source-branch to the current branch.

```
4gw19@LAPTOP-NGUM35FU MINGW64 ~/gitlab (master)
$ git cherry-pick 38e795a2849dac95b8a6da5966bfe7fd28ebface
Auto-merging file.txt
```

The selected commits were cherry-picked successfully.

The cherry-pick command was used along with the commit IDs to apply specific commits to the current branch.

Experiment 9: Analysing and Changing Git History

Git was used to view the details of a specific commit using its commit ID, including author information, date, and commit message.

```
4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (other-branch)
$ git show cc9d3a185f6eb89a0cfac1b02c5f51daf7e2b85b
commit cc9d3a185f6eb89a0cfac1b02c5f51daf7e2b85b
Author: ina528 <forstudiesalwayz@gmail.com>
Date: Tue Jan 6 10:37:04 2026 +0530

    commit 1

diff --git a/file1.txt b/file1.txt
index 2cfe96a..73a45d6 100644
--- a/file1.txt
+++ b/file1.txt
@@ -1,2 @@
-hi
+commit 1
+commit
```

The commit details such as author name, date, and message were displayed successfully.

The show command was used followed by the required commit ID. This displays the GitHub username, email ID, commit date, and the commit message.

Experiment 10: Analysing and Changing Git History

A command was written to list all commits made by the author JohnDoe between 2023-01-01 and 2023-12-31.

```
kgw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (other-branch)
$ git log --author="ina528" --since="2026-01-01" --until="2026-01-06"
commit a5f8d997fbddfdf964cfd176ddeddd19571dbc54 (HEAD -> other-branch)
Author: ina528 <forstudiesalwayz@gmail.com>
Date: Tue Jan 6 10:38:55 2026 +0530

    commit 5

commit 9ccf0bbe263bb114b17b06a3838249ccf6f6aa6e
Author: ina528 <forstudiesalwayz@gmail.com>
Date: Tue Jan 6 10:38:37 2026 +0530

    commit 4

commit fb80e9a12b6c064b6e0663a0e85c57b5a1c7805a
Author: ina528 <forstudiesalwayz@gmail.com>
Date: Tue Jan 6 10:38:14 2026 +0530

    commit 3

commit 735b44c8af96771f4accb67ff50ce88684ef5e54
Author: ina528 <forstudiesalwayz@gmail.com>
Date: Tue Jan 6 10:37:40 2026 +0530

    commit 2

commit cc9d3a185f6eb89a0cfac1b02c5f51daf7e2b85b
Author: ina528 <forstudiesalwayz@gmail.com>
Date: Tue Jan 6 10:37:04 2026 +0530

    commit 1

commit fcefe31253b94bdc57d872427d52142ebd929602
Author: ina528 <forstudiesalwayz@gmail.com>
Date: Tue Jan 6 10:36:14 2026 +0530

    commit 1
```

Commits by the specified author within the given time period were displayed successfully.

The log --author, --since, and --until options were used to filter commits based on author name and date range.

Experiment 11: Analysing and Changing Git History

The command to display the last five commits in the repository history was executed.

```
4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (master)
$ git log -5
```

```
commit a5f8d997fbddfdf964cfd176ddeddd19571dbc54 (HEAD -> other-branch)
commit a5f8d997fbddfdf964cfd176ddeddd19571dbc54 (HEAD -> other-branch)
Author: ina528 <forstudiesalwayz@gmail.com>
Date: Tue Jan 6 10:38:55 2026 +0530

    commit 5

commit 9ccf0bbe263bb114b17b06a3838249ccf6f6aa6e
Author: ina528 <forstudiesalwayz@gmail.com>
Date: Tue Jan 6 10:38:37 2026 +0530

    commit 4

commit fb80e9a12b6c064b6e0663a0e85c57b5a1c7805a
Author: ina528 <forstudiesalwayz@gmail.com>
Date: Tue Jan 6 10:38:14 2026 +0530

    commit 3

commit 735b44c8af96771f4accb67ff50ce88684ef5e54
Author: ina528 <forstudiesalwayz@gmail.com>
Date: Tue Jan 6 10:37:40 2026 +0530

    commit 2

commit cc9d3a185f6eb89a0cfac1b02c5f51daf7e2b85b
Author: ina528 <forstudiesalwayz@gmail.com>
Date: Tue Jan 6 10:37:04 2026 +0530

    commit 1
```

The most recent five commits were displayed successfully.

The log -5 command was used to list the latest five commits.

Experiment 12: Analysing and Changing Git History

The changes introduced by the commit with ID abc123 were undone.

```
4gw19@LAPTOP-NGUM35FU MINGW64 ~/OneDrive/Desktop/git-lab-clean (master)
$ git revert elec249e4834fe434412c14fc8e15e5b411cdc21
```

```
[other-branch fbebbc4] Revert "commit 4"
1 file changed, 1 insertion(+)
```

The specified commit was reverted successfully.

The revert command was used followed by the commit ID to undo the changes introduced by that commit.